Lecture Notes: Model Evaluation

CIC1205 - Machine Learning

Eduardo Bezerra

March 10, 2026

0 Contents

1 Introduction **2**
1.1 Motivation . . . 2
1.2 Estimating the Generalization Error . . . 3

2 Metrics for Classification **4**
2.1 Running Example . . . 4
2.2 The Confusion Matrix . . . 4
2.3 Accuracy . . . 5
2.4 Sensitivity, Specificity, and Precision . . . 6
2.5 The F_1 Score . . . 8
2.6 The Precision–Recall Trade-off and Curve . . . 9

3 Metrics for Regression **9**
3.1 Overview . . . 10
3.2 Running Example . . . 10
3.3 Mean Squared Error (MSE) . . . 11
3.4 Root Mean Squared Error (RMSE) . . . 11
3.5 Mean Absolute Error (MAE) . . . 12
3.6 Coefficient of Determination (R^2) . . . 13

4 Cut-off Point and Threshold Analysis **15**
4.1 From Probability Scores to Class Labels . . . 15
4.2 Effect on the Confusion Matrix . . . 15
4.3 The ROC Curve and AUC . . . 16
4.4 Selecting the Optimal Threshold . . . 16

5 The Class Imbalance Problem **18**
5.1 Why Imbalance Matters . . . 18
5.2 Appropriate Metrics and Mitigation . . . 18

6 Final Remarks **19**
6.1 Model Evaluation vs. Model Selection . . . 19
6.2 Evaluation Beyond Supervised Learning . . . 19

7 Summary **21**

1 Introduction

1.1 Motivation

When we train a machine learning model, we are searching for a hypothesis h_θ that approximates the unknown relationship between inputs $\mathbf{x}$ and targets y based on the available training data $\mathcal{D}$. A learning algorithm $\mathcal{A}$ optimizes the parameters θ according to some objective function (e.g., minimizing a loss).

Once training is complete, a central question arises:

How well will the model perform when deployed on unseen data?

This question motivates **model evaluation**: the principled assessment of how well a learned hypothesis generalizes beyond the data it was trained on.

A model that performs extremely well on training data but poorly on new examples may be suffering from **overfitting**: it has memorized patterns specific to the training set rather than learning generalizable relationships. Evaluation methods allow us to detect this problem and to compare candidate models fairly, on neutral ground.

Generalization Error

The **generalization error** of a model h_θ is the expected loss over the true (but generally unknown) data-generating distribution $p(\mathbf{x}, y)$:

$$\mathcal{E}_{\text{gen}}(h_\theta) = \mathbb{E}_{(\mathbf{x},\, y)\sim p(\mathbf{x},y)}\big[L(y,\, h_\theta(\mathbf{x}))\big].$$

This quantity is also called the *risk*, *expected loss*, or *out-of-sample error*.

Because $p(\mathbf{x}, y)$ is inaccessible, the generalization error cannot be computed exactly; it can only be **estimated** from finite held-out data.

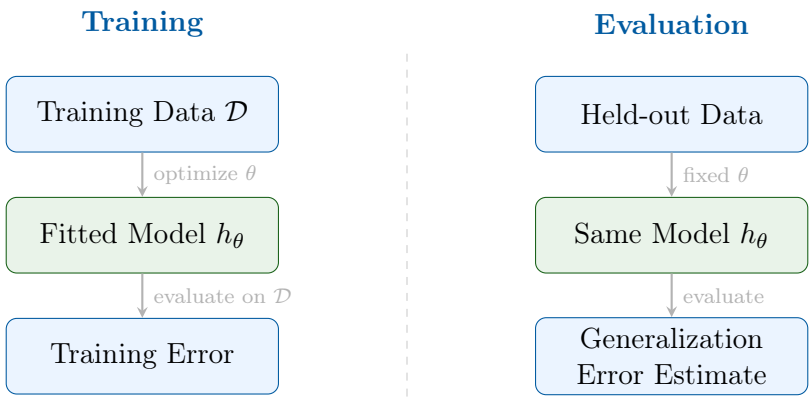

Figure 1: Separating training from evaluation. The model is fitted on training data (left); its generalization error is estimated on held-out data that played no role during training (right). A large gap between training error and held-out error is a symptom of overfitting.

Entry Ticket 1

What is the difference between **training error** and **generalization error**? Why can a model with very low training error still be considered a poor model?

1.2 Estimating the Generalization Error

Because the true distribution $p(\mathbf{x}, y)$ is inaccessible, we rely on observed data to estimate generalization ability. The key idea is to set aside one or more data subsets that are *not* used during training and to evaluate the model on those subsets.

Two questions arise when designing such procedures:

1. *How should we partition the data?*
2. *Which evaluation metrics should we use?*

Data Partitioning Strategies

The three most common strategies are summarized below.

(a) Hold-out split. The dataset is divided *once* into two or three disjoint subsets: a *training set* (used to fit the model), an optional *validation set* (used to tune hyperparameters), and a *test set* (used only for final reporting). This is simple and fast, but the resulting estimate depends heavily on which examples happen to fall in each split — a concern for small or unbalanced datasets.

(b) k-fold cross-validation. The dataset is partitioned into k equal-sized *folds*. The model is trained k times: each time, one fold plays the role of the validation set and the remaining $k-1$ folds are used for training. The k validation scores are then averaged to produce a single, more stable estimate. A typical choice is $k = 5$ or $k = 10$. This strategy uses the data more efficiently and yields a lower-variance estimate than a single hold-out split, at the cost of k times the computation.

(c) Nested cross-validation. When we also need to *select* hyperparameters (and not just evaluate a fixed model), a single cross-validation loop is insufficient — the fold that validates model selection is no longer a truly "unseen" test set. Nested cross-validation solves this with two loops: an *outer* loop that holds out a fold for unbiased final evaluation, and an *inner* loop that runs another round of cross-validation *inside* the remaining folds to select hyperparameters. Because the inner loop never touches the outer test fold, the outer estimate remains unbiased even when hyperparameters are tuned.

Looking ahead: Model Selection

These lectures notes focus on *evaluating* a fixed trained model. A closely related but distinct problem, *choosing* among models or hyperparameter configurations, is called **model selection**. Both activities rely on the same partitioning strategies above, but they raise additional questions: How do we compare models fairly? How do we avoid *data leakage* when the same data is reused for selection and evaluation? These questions will be the subject of the next set of lecture notes.

Choosing evaluation metrics

The answer to the second question — which metric to use — depends on the learning task. Classification and regression require fundamentally different metrics, which are covered in the sections that follow. No single metric tells the whole story; reporting multiple complementary measures is good practice.

2 Metrics for Classification

2.1 Running Example

To make each metric concrete, we will use a single running example throughout this section.

Scenario. A classifier is trained to detect whether an email is **spam** (positive class, label = 1) or **not spam** (negative class, label = 0). The model is evaluated on a test set of **30 emails**. The table below shows the full confusion matrix obtained at a fixed decision threshold $\alpha = 0.5$:

Actual \ *Predicted*	Spam ($\hat{y} = 1$)	Not spam ($\hat{y} = 0$)	Total
Spam ($y = 1$)	**TP = 8**	**FN = 3**	11
Not spam ($y = 0$)	**FP = 2**	**TN = 15**	17
Total	10	18	30

All metrics in this section will be computed from these four numbers: $TP = 8$, $FN = 3$, $FP = 2$, $TN = 15$.

2.2 The Confusion Matrix

The **confusion matrix** is the foundation of classification evaluation. Rather than collapsing performance into a single number, it reveals the *type* of errors the model makes.

- **TP (True Positive)**: the model correctly predicted "positive" when the true label was positive. In our example, 8 actual spam emails were correctly flagged.
- **TN (True Negative)**: the model correctly predicted "negative" when the true label was negative. Here, 15 legitimate emails were correctly left alone.
- **FP (False Positive, Type I error)**: the model predicted "positive" but the true label was negative. Here, 2 legitimate emails were wrongly flagged as spam.
- **FN (False Negative, Type II error)**: the model predicted "negative" but the true label was positive. Here, 3 spam emails were missed.

These two types of errors have very different consequences depending on the application: in spam filtering, a FP means a legitimate email is deleted; a FN means spam gets through. In disease screening, a FP means an unnecessary biopsy; a FN means a disease goes undetected.

	Predicted Positive	Predicted Negative
Actual Positive	**TP = 8** Spam correctly flagged	**FN = 3** Spam missed
Actual Negative	**FP = 2** Ham wrongly flagged	**TN = 15** Ham correctly kept

Figure 2: Confusion matrix for the spam-detection running example (30 emails, $\alpha = 0.5$). Green cells are correct predictions; red cells are errors. FP and FN represent qualitatively different failure modes with different costs depending on the application.

Entry Ticket 2

A medical test for a disease produces the following results on 200 patients: $TP = 45$, $FP = 10$, $FN = 5$, $TN = 140$.

1. Write the confusion matrix and compute the total number of errors.
2. Which type of error (FP or FN) is more dangerous in this clinical context? Why?
3. In a spam filter, which type of error is typically more annoying to a user?

2.3 Accuracy

The most intuitive summary metric is **accuracy**: the overall fraction of correctly classified examples.

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}.$$

Computed on the running example.

$$\text{Accuracy} = \frac{8 + 15}{8 + 2 + 3 + 15} = \frac{23}{28} \approx 0.821.$$

The model classifies 82.1 % of emails correctly.

When accuracy misleads. Accuracy is easy to interpret but can be dangerously optimistic on **imbalanced datasets**. Suppose that in a different scenario only 5 % of emails are actually spam. A trivial classifier that *always* predicts "not spam" achieves

$$\text{Accuracy}_{\text{trivial}} = \frac{0 + 950}{1000} = 95\,\%,$$

despite detecting *zero* spam emails. A genuine classifier that correctly catches 80 % of spam but also misclassifies some legitimate mail might score only 88 %, and yet be vastly more useful. Accuracy, in this case, rewards inaction.

5 % spam dataset: accuracy does not capture usefulness

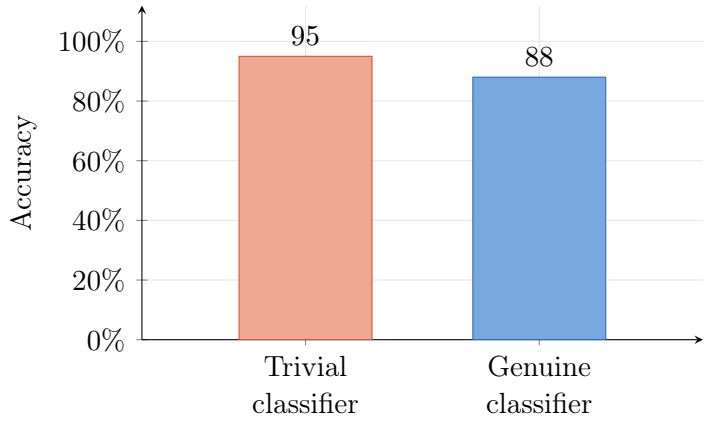

Figure 3: On a dataset with 5 % spam, a trivial "always-negative" classifier scores 95 % accuracy, outperforming a genuine classifier (88 %) despite making zero correct spam detections. This illustrates why accuracy alone is insufficient for imbalanced problems.

Entry Ticket 3

A dataset contains 990 legitimate transactions and 10 fraudulent ones. A model always predicts "legitimate."

1. Compute its accuracy.
2. How many fraudulent transactions does it detect? What does this tell you?
3. What alternative metric would better reveal its failure?

2.4 Sensitivity, Specificity, and Precision

To capture the two distinct types of error separately, we define three finer-grained metrics.

Sensitivity (Recall)

$$\text{Sensitivity} = \text{Recall} = \frac{TP}{TP + FN}.$$

"Of all actual positives, how many did the model detect?"

High sensitivity means few false negatives. It answers the question from the perspective of the positive class: how completely does the model identify it? This is critical in applications where *missing* a positive is very costly (e.g., disease screening, fraud detection).

Specificity

$$\text{Specificity} = \frac{TN}{TN + FP}.$$

"Of all actual negatives, how many were correctly rejected?"

High specificity means few false positives. It answers the question from the perspective of the negative class: how well does the model avoid raising false alarms? This matters in applications where *false alarms* are costly (e.g., legal evidence, unnecessary surgery).

Precision

$$\text{Precision} = \frac{TP}{TP + FP}.$$

"When the model predicts positive, how often is it correct?"

High precision means reliable positive predictions. It answers the question from the perspective of the predictions: among all examples the model labeled positive, how many truly are? This is important when acting on a positive prediction is expensive or irreversible.

Computed on the running example.

$$\text{Sensitivity} = \frac{8}{8+3} = \frac{8}{11} \approx 0.727$$

$$\text{Specificity} = \frac{15}{15+2} = \frac{15}{17} \approx 0.882$$

$$\text{Precision} = \frac{8}{8+2} = \frac{8}{10} = 0.800$$

Interpretation: the model correctly catches 72.7 % of actual spam emails (sensitivity), correctly ignores 88.2 % of legitimate emails (specificity), and when it does flag an email as spam, it is right 80 % of the time (precision).

Sensitivity vs. Precision: different denominators

A common source of confusion is that sensitivity and precision both involve TP in the numerator, yet measure very different things.

- **Sensitivity** normalizes by all actual positives ($TP + FN$): it measures *coverage* of the positive class.
- **Precision** normalizes by all predicted positives ($TP + FP$): it measures *reliability* of positive predictions.

A model can have high sensitivity and low precision (it catches everything but produces many false alarms) or high precision and low sensitivity (its predictions are reliable but it misses many positives).

Entry Ticket 4

Using the running example values ($TP = 8$, $FN = 3$, $FP = 2$, $TN = 15$):

1. Verify the three computed values above by redoing the calculations yourself.
2. Suppose the system manager decides that wrongly deleting a legitimate email is twice as bad as letting a spam through. Which metric captures the first

error? Which captures the second?

2.5 The F_1 Score

Because precision and recall are often in tension, it is useful to combine them into a single number. The F_1 **score** is the *harmonic mean* of precision and recall:

F_1 Score

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\,TP}{2\,TP + FP + FN}.$$

Why harmonic mean and not arithmetic mean? Consider a degenerate model that always predicts "positive". It achieves Recall $= 1$ (it catches every positive) but Precision ≈ 0 (almost all its predictions are wrong). The arithmetic mean would give $(1+0)/2 = 0.5$, which sounds reasonable. The harmonic mean gives $F_1 = 2 \cdot (1 \cdot 0)/(1 + 0) = 0$, correctly signaling a useless model. In general, the harmonic mean is pulled strongly toward the *lower* of the two values, so a model only achieves a high F_1 when *both* metrics are high.

Computed on the running example.

$$F_1 = 2 \times \frac{0.800 \times 0.727}{0.800 + 0.727} = 2 \times \frac{0.582}{1.527} \approx 0.762.$$

Alternatively, using the raw counts:

$$F_1 = \frac{2 \times 8}{2 \times 8 + 2 + 3} = \frac{16}{21} \approx 0.762.$$

The F_β family. When we want to weight recall relative to precision, we can use the general F_β score:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \times \text{Recall}}{(\beta^2 \times \text{Precision}) + \text{Recall}}.$$

- $\beta > 1$ places more weight on recall: use this when *missing positives* is more costly than raising false alarms (e.g., F_2 in medical diagnosis).
- $\beta < 1$ places more weight on precision: use this when *false alarms* are more costly than missed detections (e.g., $F_{0.5}$ in spam filtering where deleting legitimate mail is unacceptable).
- $\beta = 1$ recovers the standard F_1 score (equal weight to both).

Entry Ticket 5

Using the running example ($TP = 8$, $FP = 2$, $FN = 3$, $TN = 15$):

1. You computed Precision ≈ 0.800 and Recall ≈ 0.727 earlier. Verify that $F_1 \approx 0.762$ using the harmonic-mean formula.
2. Now suppose the email provider decides that falsely flagging a legitimate email

Harmonic mean (F_1) vs. arithmetic mean when $P = 0.8$

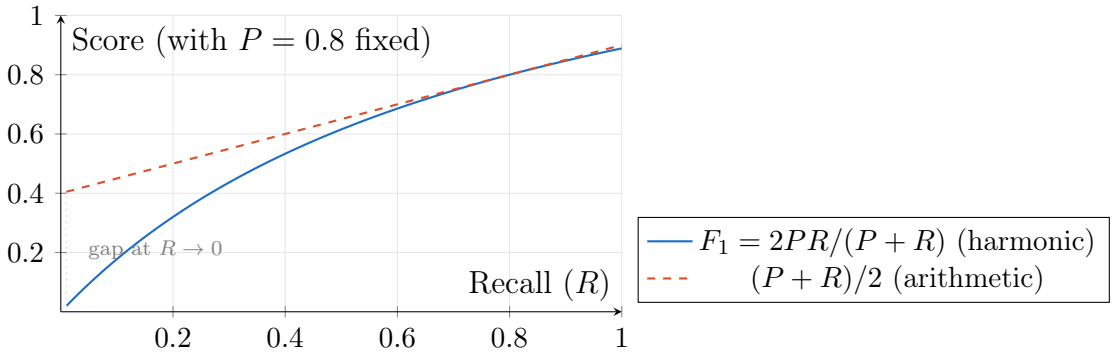

Figure 4: F_1 (blue) vs. arithmetic mean (red) as a function of recall when precision is fixed at 0.8. The harmonic mean stays near zero when recall is low, while the arithmetic mean gives an inflated score. Only when both P and R are high does F_1 become high.

as spam is three times worse than missing a spam. Should they use F_β with $\beta > 1$ or $\beta < 1$? Explain.

2.6 The Precision–Recall Trade-off and Curve

All the metrics above depend on the choice of decision threshold α. A natural question is: *how does the model's precision–recall behavior look across all possible thresholds?*

As α varies from 0 to 1, every example eventually flips from predicted positive to predicted negative. At each value of α, we can compute (Precision, Recall) and plot the resulting pair as a single point. Connecting all such points traces the **Precision–Recall (PR) curve**.

Interpreting the curve.

- At $\alpha = 0$, every example is classified as positive: Recall $= 1$ but Precision $=$ (fraction of true positives in the data), which is low for imbalanced datasets.
- At $\alpha = 1$, nothing is classified as positive: Precision is undefined (or set to 1 by convention) and Recall $= 0$.
- A model with a curve that *hugs the upper-right corner* maintains high precision even at high recall; it is a strong classifier.

The **area under the PR curve (PR-AUC)** summarizes overall performance across all thresholds. PR-AUC is particularly informative for imbalanced datasets, where the ROC curve (see Section 4) can give an overly optimistic picture.

3 Metrics for Regression

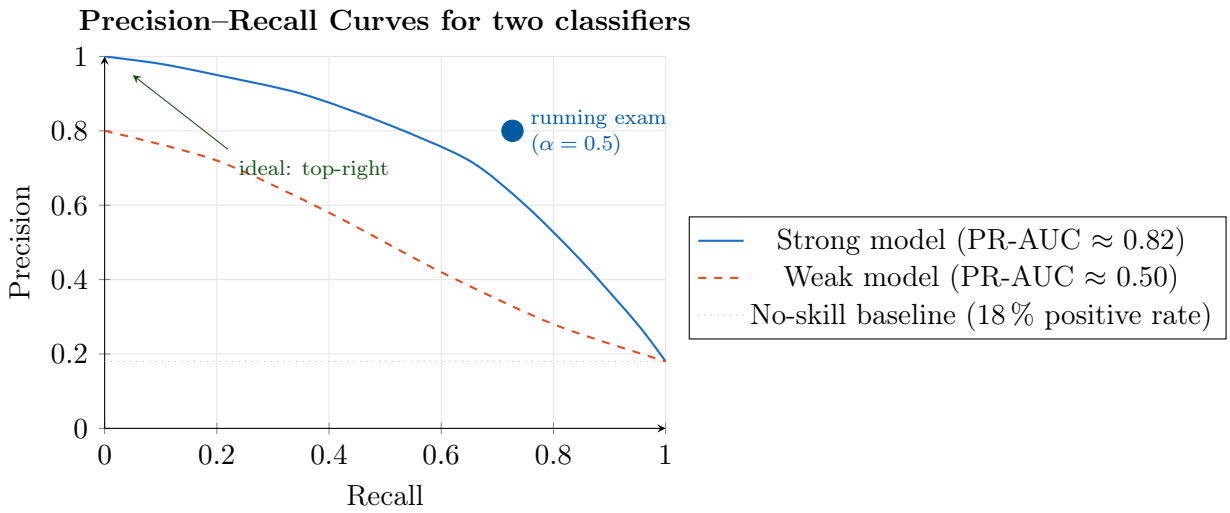

Figure 5: PR curves for two classifiers. Each point on the curve corresponds to a threshold α. The filled dot marks the operating point of the running example at $\alpha = 0.5$: Recall $\approx$ 0.727, Precision $=$ 0.800. A stronger model's curve lies above and to the right; the dotted baseline corresponds to a random (no-skill) classifier.

3.1 Overview

While classification models predict discrete labels, **regression models** predict continuous numerical values. Evaluation metrics for regression quantify the **magnitude of prediction errors**.

Let m be the number of test examples, $y^{(i)}$ the true target, and $\hat{y}^{(i)} = h_\theta(\mathbf{x}^{(i)})$ the model's prediction. The **residual** for example i is

$$e^{(i)} = y^{(i)} - \hat{y}^{(i)}.$$

A positive residual means the model under-predicted; a negative residual means it over-predicted. MSE, RMSE, and MAE all aggregate these residuals in different ways.

3.2 Running Example

To ground the regression metrics, we use the following small dataset of 5 test examples, where a model predicts the daily temperature (in °C) from weather features:

Example	True $y^{(i)}$	Predicted $\hat{y}^{(i)}$	Residual $e^{(i)}$	$(e^{(i)})^2$
1	20	18	+2	4
2	25	27	−2	4
3	30	28	+2	4
4	15	14	+1	1
5	22	30	−8	64
			Sum	**77**

Example 5 contains a large error (-8°C). We will see how differently the metrics treat this outlier.

Residuals: vertical distance between true value and prediction

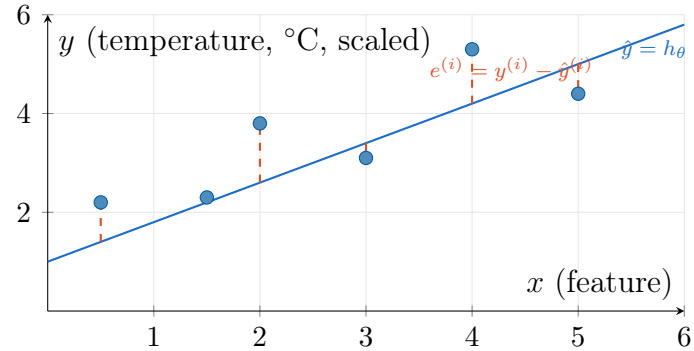

Figure 6: Each red dashed segment is the residual for one training example; the vertical distance between the observed value and the model's prediction. MSE squares these distances; MAE takes their absolute values; both are then averaged over all examples.

3.3 Mean Squared Error (MSE)

Mean Squared Error

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}\left(y^{(i)} - \hat{y}^{(i)}\right)^2.$$

MSE squares each residual before averaging, which **penalizes large errors disproportionately**. An error of 8 contributes 64 to the sum, while four errors of 2 together contribute only $4 \times 4 = 16$.

Computed on the running example.

$$\text{MSE} = \frac{4+4+4+1+64}{5} = \frac{77}{5} = 15.4\ (°\text{C})^2.$$

Note that MSE is expressed in **squared units** (here, °C^2), which makes it difficult to interpret directly in the original scale.

3.4 Root Mean Squared Error (RMSE)

Root Mean Squared Error

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(y^{(i)} - \hat{y}^{(i)}\right)^2}.$$

Taking the square root restores the metric to the **same units as the target variable**, making it directly interpretable.

Computed on the running example.

$$\text{RMSE} = \sqrt{15.4} \approx 3.92\ ^\circ\text{C}.$$

Interpretation: *on average, the model's temperature predictions are off by about 3.9°C*, with larger errors weighted more heavily.

3.5 Mean Absolute Error (MAE)

Mean Absolute Error

$$\text{MAE} = \frac{1}{m}\sum_{i=1}^{m}\left|y^{(i)} - \hat{y}^{(i)}\right|.$$

MAE takes the absolute value instead of squaring, so each residual contributes in proportion to its magnitude. This makes MAE **more robust to outliers**.

Computed on the running example.

$$\text{MAE} = \frac{|2| + |-2| + |2| + |1| + |-8|}{5} = \frac{2+2+2+1+8}{5} = \frac{15}{5} = 3.0\ ^\circ\text{C}.$$

Interpretation: *on average, the model's predictions deviate from the true temperature by 3.0°C*, treating all errors equally regardless of sign or magnitude.

Comparison: MSE/RMSE vs. MAE. The difference between RMSE (≈ 3.92) and MAE (3.0) is large here because of the outlier in Example 5 (error of -8). If we removed that example, both metrics would be much closer. This illustrates the key difference:

- **MSE/RMSE**: strongly influenced by large errors; use them when large deviations are particularly unacceptable (e.g., aircraft fuel estimation).
- **MAE**: treats all errors uniformly; use it when a robust summary of typical error size is needed (e.g., daily temperature forecast quality).

The inequality RMSE $\geq$ MAE

It is always true that RMSE $\geq$ MAE ≥ 0. The two are equal only when all residuals have the same absolute value. The larger the spread among residuals (i.e., the more some errors dominate), the larger the gap between RMSE and MAE. This gap is therefore informative in itself.

Entry Ticket 6

Using the running example (temperature predictions):

1. Remove Example 5 (the outlier) and recompute MSE, RMSE, and MAE for the remaining 4 examples. How much do the values change?
2. This shows that RMSE is more sensitive to the outlier than MAE. In which real-world applications would you *not* want to ignore large errors? In which applications would you prefer a metric that does not over-penalize them?

Penalty function: MSE vs. MAE

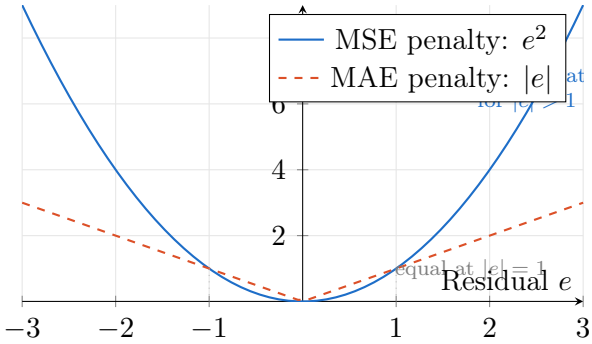

Figure 7: Penalty as a function of residual for MSE (blue) and MAE (red dashed). For residuals with $|e| < 1$, MAE penalizes more than MSE. For $|e| > 1$, MSE penalizes more steeply, amplifying the influence of large errors.

3.6 Coefficient of Determination (R^2)

MSE, RMSE, and MAE all measure the *absolute* magnitude of errors, but they depend on the scale and variability of the target variable. A model predicting house prices in millions of dollars may have MAE = 50,000, which sounds large but might be excellent.

The **coefficient of determination** R^2 addresses this by measuring error relative to a *baseline*: how much better is the model compared to simply predicting the mean $\bar{y}$ for every example?

Coefficient of Determination

$$R^2 = 1 - \frac{\mathrm{SS}_{\mathrm{res}}}{\mathrm{SS}_{\mathrm{tot}}},$$

where

$$\mathrm{SS}_{\mathrm{res}} = \sum_{i=1}^{m} (y^{(i)} - \hat{y}^{(i)})^2 \quad \text{(residual sum of squares: unexplained variation)}$$

$$\mathrm{SS}_{\mathrm{tot}} = \sum_{i=1}^{m} (y^{(i)} - \bar{y})^2 \quad \text{(total sum of squares: total variation around the mean).}$$

Interpreting R^2. The ratio $\mathrm{SS}_{\mathrm{res}}/\mathrm{SS}_{\mathrm{tot}}$ is the *fraction of variance that the model fails to explain*. Subtracting it from 1 gives the fraction it *does* explain:

- $R^2 = 1$: $\mathrm{SS}_{\mathrm{res}} = 0$; perfect fit (all predictions are exact).
- $R^2 = 0$: $\mathrm{SS}_{\mathrm{res}} = \mathrm{SS}_{\mathrm{tot}}$; the model explains nothing; it does no better than predicting $\bar{y}$ for every example.
- $R^2 < 0$: the model is *worse* than the mean predictor; this can happen if the model is very poorly specified.

Computed on the running example. The mean of the true values is $\bar{y} = (20 + 25 + 30 + 15 + 22)/5 = 112/5 = 22.4$.

Example	$y^{(i)}$	$(y^{(i)} - \hat{y}^{(i)})^2$	$(y^{(i)} - \bar{y})^2$
1	20	4	$(20 - 22.4)^2 = 5.76$
2	25	4	$(25 - 22.4)^2 = 6.76$
3	30	4	$(30 - 22.4)^2 = 57.76$
4	15	1	$(15 - 22.4)^2 = 54.76$
5	22	64	$(22 - 22.4)^2 = 0.16$
Sum		**77**	**125.2**

$$R^2 = 1 - \frac{77}{125.2} \approx 1 - 0.615 = 0.385.$$

The model explains about 38.5 % of the variance in temperature, which is modest, primarily because of the large error in Example 5.

R^2: model residuals vs. deviations from the mean

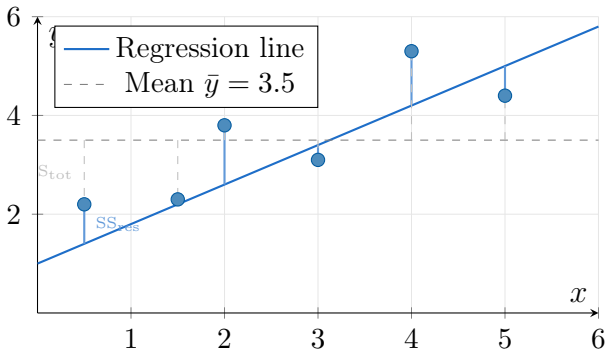

Figure 8: R^2 compares two sets of vertical distances: blue solid segments show the residuals from the fitted model (SS_{res}), and gray dashed segments show the deviations from the mean $\bar{y}$ (SS_{tot}). R^2 measures the fractional reduction in these distances achieved by the model.

Entry Ticket 7

A regression model predicts students' exam scores (out of 100) from hours studied. On a test set of 4 students:

Student	True y	Predicted $\hat{y}$
A	70	68
B	80	83
C	60	59
D	90	80

1. Compute MAE and RMSE.
2. Compute $\bar{y}$, then SS_{res} and SS_{tot}, and finally R^2.
3. In plain language, what does the R^2 value tell a teacher reading this report?

4 Cut-off Point and Threshold Analysis

4.1 From Probability Scores to Class Labels

Many classifiers do not output a hard label directly. Instead, they output a **probability score** $\hat{p} = \widehat{\Pr}(\text{class} = 1 \mid \mathbf{x})$. Examples include logistic regression (via the sigmoid function), decision tree ensembles (calibrated fraction of trees voting positive), and neural networks with softmax output.

To convert the score into a class prediction, we apply a **decision threshold** α:

Cut-off Point / Decision Threshold

A cut-off point $\alpha \in (0, 1)$ converts a probability score into a binary prediction:

$$\hat{y} = \begin{cases} 1 & \text{if } \hat{p} \geq \alpha, \\ 0 & \text{otherwise.} \end{cases}$$

The default choice is $\alpha = 0.5$, but **the optimal value depends on the application and the cost of each type of error**.

4.2 Effect on the Confusion Matrix

Let us trace what happens to the running-example confusion matrix as α changes. Suppose we sort the 30 test emails by their probability score (highest to lowest). Moving the threshold α is equivalent to drawing a vertical cut in this sorted list: emails to the left are predicted positive, emails to the right are predicted negative.

	TP	**FP**	**FN**	**TN**	**Precision / Recall**
Moderate $\alpha = 0.5$ (baseline)	8	2	3	15	0.800 / 0.727
High $\alpha = 0.8$ (conservative)	6	1	5	18	0.857 / 0.545
Low $\alpha = 0.3$ (aggressive)	10	5	1	14	0.667 / 0.909

The pattern is clear:

- **Raising** α makes the model more *conservative*: it predicts positive less often. Precision increases (the positive predictions are more reliable), but recall decreases (more positives are missed).
- **Lowering** α makes the model more *aggressive*: it predicts positive more often. Recall increases (fewer positives are missed), but precision decreases (more false alarms are raised).

This is the fundamental **precision–recall trade-off**, and it is controlled entirely by the threshold α.

4.3 The ROC Curve and AUC

An alternative way to summarize threshold behavior is the **Receiver Operating Characteristic (ROC) curve**, which plots the True Positive Rate (TPR) against the False Positive Rate (FPR) as α varies:

$$\text{TPR} = \frac{TP}{TP + FN} = \text{Recall}, \qquad \text{FPR} = \frac{FP}{FP + TN} = 1 - \text{Specificity}.$$

While the PR curve focuses on the positive class, the ROC curve frames the trade-off differently: *how many true positives can I capture (TPR) for each fraction of false positives I am willing to tolerate (FPR)?*

Area Under the ROC Curve (AUC)

The **AUC** (also called AUC-ROC) is the area under the ROC curve. It has an important probabilistic interpretation:

$$\text{AUC} = \Pr\big(\hat{p}(\mathbf{x}^+) > \hat{p}(\mathbf{x}^-)\big),$$

where $\mathbf{x}^+$ is a randomly chosen positive example and $\mathbf{x}^-$ is a randomly chosen negative example. In other words, **AUC measures the probability that the model assigns a higher score to a positive than to a negative**.

- AUC $= 1.0$: perfect discrimination (model always ranks positives above negatives).
- AUC $= 0.5$: random classifier (no discrimination ability).
- AUC < 0.5: the model is systematically wrong (rare in practice).

4.4 Selecting the Optimal Threshold

There is no universally optimal α; the right choice depends on the cost structure of the application. Common strategies include the following.

Maximize a target metric. Sweep α over the validation set and choose the value that maximizes the metric of interest (e.g., F_1, F_β, or the Youden index $J = \text{TPR} - \text{FPR}$, which balances sensitivity and specificity).

Use a cost matrix. When the cost of FP and FN can be expressed in monetary or clinical terms, we can compute the expected cost at each α and choose the minimizer. For example, if a false negative costs \$1,000 (undetected fraud) and a false positive costs \$50 (unnecessary investigation), the optimal α should lean toward lower values to reduce FN.

Operational constraints. In many deployed systems, the threshold is set to satisfy a business requirement, such as "review at most 200 cases per day" or "achieve at least 95 % recall on the positive class." These constraints translate directly into a target TPR or FPR, which can be read off the ROC curve.

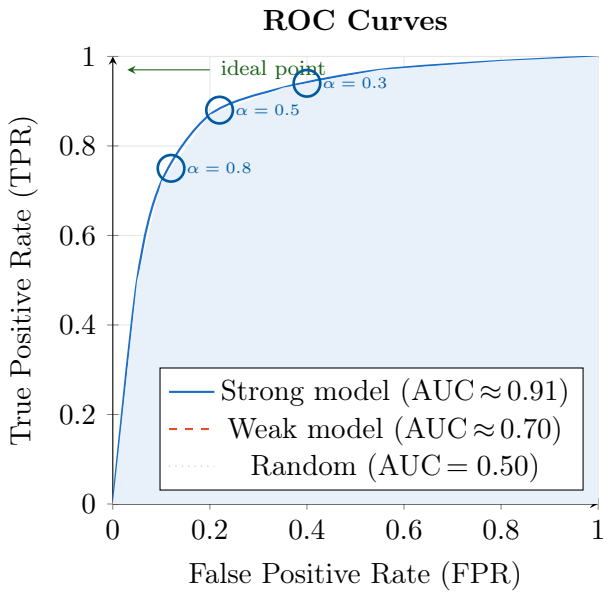

Figure 9: ROC curves for a strong and a weak classifier. Each point on the curve corresponds to a specific threshold α; three representative thresholds are marked on the strong model. The shaded area under the blue curve is the AUC. As α decreases (moving right along the curve), both TPR and FPR increase: we catch more positives but also raise more false alarms.

ROC curve vs. PR curve: when to use which?

Both curves sweep α from 0 to 1, but they emphasize different aspects of performance.

- **ROC curve**: uses FPR in the x-axis, which involves TN. When the negative class is very large (imbalanced dataset), TN is large and FPR stays small even if many false positives are produced. The ROC curve therefore gives an *optimistically distorted* view on imbalanced data.
- **PR curve**: does not involve TN at all. It focuses entirely on the positive class, making it more sensitive to the model's behavior on rare positives. **Prefer the PR curve when positive examples are rare.**

Entry Ticket 8

A classifier is trained to detect fraudulent bank transactions (0.1 % of all transactions are fraudulent).

1. Should you prefer the ROC curve or the PR curve to evaluate this model? Justify your answer referring to which cell of the confusion matrix makes the ROC curve misleading in this setting.
2. Suppose the bank sets the cost of a false negative (missed fraud) at \$500 and the cost of a false positive (blocked legitimate transaction) at \$5. Should the bank choose a high or a low threshold α? Describe qualitatively how this affects TPR and FPR.

5 The Class Imbalance Problem

5.1 Why Imbalance Matters

Many real-world datasets are **highly imbalanced**: the positive class (fraud, disease, defect) occurs much less often than the negative class. Table 1 shows realistic proportions in three common applications.

Application	Positive rate	Trivial accuracy
Credit card fraud	$\approx 0.1\,\%$	$99.9\,\%$
Rare disease screening	$\approx 1\,\%$	$99.0\,\%$
Spam filtering	$\approx 20\,\%$	$80.0\,\%$

Table 1: Positive class rate and the accuracy of a trivial "always negative" classifier in three imbalanced settings. In fraud detection, a completely useless model achieves $99.9\,\%$ accuracy.

Beyond misleading accuracy, imbalance creates additional challenges:

- **Default threshold is suboptimal.** At $\alpha = 0.5$, the model has seen few positive examples during training, so its scores for positives tend to be lower than for a balanced dataset. The default threshold often sits in a region with very low recall.
- **Misleading model comparisons.** Two models may have the same accuracy yet differ substantially in recall: one may catch $0\,\%$ of positives and the other $60\,\%$.
- **Training instability.** The loss function may be dominated by the majority class, leading the model to effectively "ignore" the minority class.

5.2 Appropriate Metrics and Mitigation

Metric choice. For imbalanced problems, the following metrics are more informative than plain accuracy: F_β (especially F_1 or F_2), precision–recall AUC, and (with care) AUC-ROC. Raw accuracy should be avoided as the primary evaluation criterion.

Mitigation strategies. When training on imbalanced data, practitioners often combine appropriate metrics with data-level or algorithm-level adjustments:

- **Oversampling the minority class**: duplicate or synthetically generate positive examples (e.g., SMOTE, short for Synthetic Minority Oversampling Technique).
- **Undersampling the majority class**: randomly remove negative examples until the classes are more balanced.
- **Class-weighted loss**: assign a higher penalty to errors on the minority class in the training objective (e.g., `class_weight='balanced'` in scikit-learn).
- **Threshold adjustment**: after training on imbalanced data, lower α to increase recall on the minority class.

Entry Ticket 9

A dataset for predicting loan default has 950 non-default examples and 50 default examples.

1. A model always predicts "no default." What is its accuracy? What is its recall for the default class?
2. Why would using F_1 score instead of accuracy expose this model's failure?
3. A second model achieves accuracy 91 %, recall 70 %, and precision 65 % for defaults. Which model would you deploy? Justify using the metrics above.

6 Final Remarks

6.1 Model Evaluation vs. Model Selection

Model evaluation is closely related to, but distinct from, **model selection**.

- **Model evaluation** asks: *"How good is this trained model on unseen data?"* Its purpose is to estimate the generalization error of a fixed, trained model.
- **Model selection** asks: *"Which model or hyperparameter configuration should we use?"* Its purpose is to choose among alternatives by comparing their generalization performance.

The key practical point is that **these two activities require separate data**. When we tune hyperparameters on a validation set, the selected model is adapted to that set. If we then *also* use the validation set to report final performance, we obtain an optimistically biased estimate; the model has effectively "seen" the validation labels through the selection process. This phenomenon is called **data leakage**.

The correct workflow separates three data pools:

1. **Training set**: used to fit the model parameters.
2. **Validation set** (or inner cross-validation folds): used to tune hyperparameters and select among model variants.
3. **Test set**: held out completely and touched only *once*, at the end, to report the final unbiased estimate of generalization error.

Using the test set more than once, or using it to guide any decision, invalidates its role as an unbiased evaluator.

6.2 Evaluation Beyond Supervised Learning

Unsupervised learning poses additional challenges because there are no true labels against which to compare predictions. Different problem types use the following families of metrics.

Clustering. The goal is to find groups of similar examples. In the absence of labels, quality is measured by *compactness* (examples within a cluster should be similar) and *separation* (different clusters should be distinct):

- *Silhouette Score* (-1 to $+1$): compares the average distance within a cluster to the average distance to the nearest other cluster. Higher is better.
- *Davies–Bouldin Index*: average ratio of within-cluster spread to between-cluster distance. Lower is better.

Dimensionality reduction. Methods such as PCA and autoencoders are evaluated by *reconstruction error*: how well can the original data be recovered from the low-dimensional representation? Methods such as t-SNE and UMAP are evaluated by how well they preserve neighborhood relationships from the high-dimensional space.

Density estimation. Models that learn the data distribution (e.g., GMMs, VAEs) are evaluated by *log-likelihood* on held-out data: a higher log-likelihood means the model assigns higher probability to the observed test points.

Entry Ticket 10

Explain in your own words why model selection requires held-out data that is *separate* from the test set used for final evaluation. Describe a concrete scenario in which using the same data for both leads to an overly optimistic performance report.

7 Summary

Concept	**Key idea / formula**
Generalization error	$\mathcal{E}_{\text{gen}} = \mathbb{E}[L(y, h_\theta(\mathbf{x}))]$; estimated from held-out data never used in training
Hold-out / k-fold	Strategies for partitioning data; k-fold gives lower-variance estimates
Confusion matrix	Cross-table of TP, FP, FN, TN; reveals types of errors, not just total errors
Accuracy	$(TP+TN)/(TP+FP+FN+TN)$; misleading when classes are imbalanced
Sensitivity (Recall)	$TP/(TP+FN)$; coverage of the positive class; penalizes missed positives
Specificity	$TN/(TN+FP)$; coverage of the negative class; penalizes false alarms
Precision	$TP/(TP+FP)$; reliability of positive predictions
F_1 score	Harmonic mean of precision and recall; F_β generalizes with weight β
MSE / RMSE	Quadratic error; sensitive to outliers; RMSE in same units as target
MAE	Linear error; robust to outliers; average absolute deviation
R^2	Fraction of variance explained; $R^2 = 1$ perfect, $R^2 = 0$ mean baseline
Cut-off point α	Threshold on probability score; controls precision–recall trade-off
ROC / AUC	Threshold-independent discriminative performance; AUC = Pr(rank pos. above neg.)
PR curve / PR-AUC	Preferred over ROC for imbalanced datasets (does not use TN)

Key Takeaways

1. Model evaluation estimates generalization performance on data *never seen* during training; training error alone is not a reliable indicator of quality.
2. The confusion matrix is the foundation of classification evaluation; all metrics derive from TP, FP, FN, and TN, which capture qualitatively different failure modes.
3. Accuracy is intuitive but misleading on imbalanced datasets; always comple-

ment it with precision, recall, F_β, or PR-AUC in such settings.

4. For regression, report both an absolute error (MAE or RMSE) and a relative measure (R^2) to characterize both magnitude and explained variance.
5. The decision threshold α directly controls the precision–recall trade-off; the optimal value depends on the cost of FP and FN in the specific application.
6. Model selection and model evaluation are distinct activities that require separate data; reusing the test set for selection introduces data leakage.